

CS 3600 Intro to Artificial Intelligence

Homework 3

Bayesian Classifiers and Hidden Markov Models

This assignment has two distinct parts:

- Creating a Naive Bayesian classifier
- Estimating the state of the hidden variable based on a series of emissions

You will be working in one directory, but these are two completely separate projects.

This homework is due at 11:59 pm on Sunday, February 8th. It must be uploaded to Gradescope by then. No credit will be given for late submissions.

1 Naive Bayes Classifier

1.1 The Problem

You work for the Drug Enforcement Agency. Agents have noticed that some retail spaces with laundromat signs in front are actually depots where illegal substances are distributed.

By staking out and raiding randomly chosen laundromats across the country, agents have discovered the following:

- About 1 in 5 alleged laundromats are actually drug depots.
- The average number of visitors per hour to a drug depot seems to be higher than the average for actual laundromats.
- The floor space (in square feet) of drug depot is often smaller than the floor space of a laundromat.
- If a shop is a drug depot, it is more likely to be owned by a convicted felon than if it is a legit laundromat.

In the `data` directory, you will find five CSV files.

- `depots.csv` - data from some of the random raids that turned out to be drug depots
- `laundromats.csv` - data from some of the random raids that turned out to be actual laundromats
- `unlabeled_validate.csv` - Unlabeled data that you can validate your system on.
- `labels_validate.csv` - The labels that go with `unlabeled_validate.csv`

Except for the label file, each of these files has five columns:

- `shop_id` - a non-negative integer that is unique for each shop.
- `area` - a positive floating point number that gives the square footage of the shop based on the building plans on file with the municipality.
- `visitors1` - a non-negative integer representing the number of visitors to the shop counted during a randomly chosen one-hour stake-out.
- `visitors2` - a non-negative integer representing the number of visitors to the shop counted during a second randomly chosen one-hour stake-out.
- `felon` - 1 if the shop is owned by a convicted felon; 0 if not.

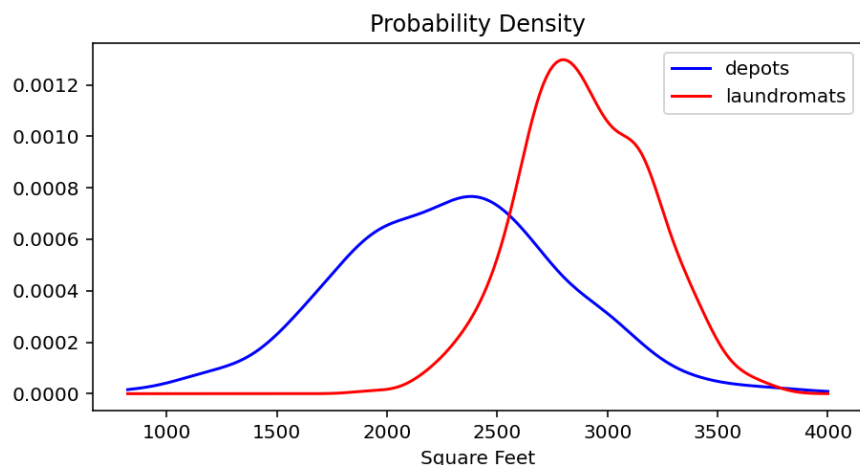
The `labels_validate.csv` has two columns:

- `shop_id` - a non-negative integer that is unique for each shop.
- `oneof` - Either `laundromats` or `depots`.

The first step is to compute some parameters from the labeled data. You will be implementing the missing code in `make_parameters.py`.

1.1.1 Modeling Square Footage

If you plot a kernel density estimate of the areas in `depots.csv` or `laundromats.csv`, you will find that each probability density function seems to be a normal distribution. However each has its own mean and variance.



(Note that there is a lot of overlap – a classifier based only on the square footage would not be accurate.)

The first step is use `depots.csv` and `laundromats.csv` and to compute some parameters:

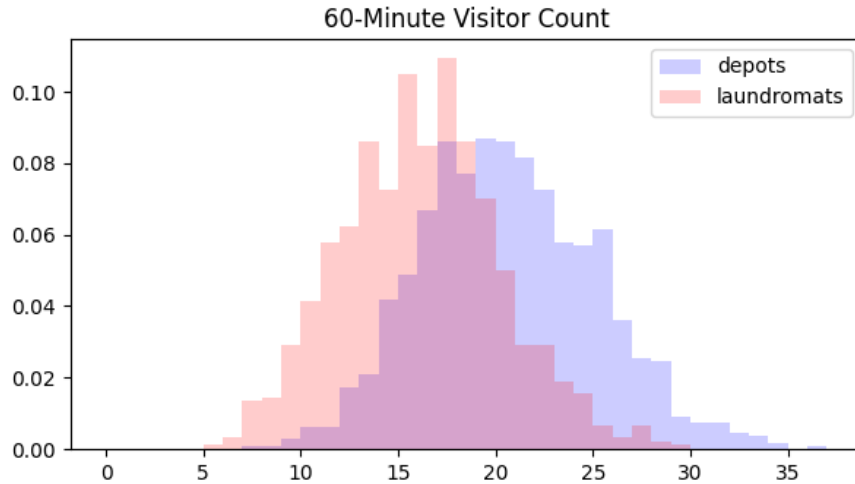
What are the values for μ_D and σ_D that would make the observed area data in `depots.csv` most likely?

What are the values for μ_L and σ_L that would make the observed area data in `laundromats.csv` most likely?

(Quick note: JAX makes it easy to compute the *variance* (σ^2) of the data in an array. Scipy makes it easy to compute the log probability of a value given the *standard deviation* (σ). Be careful.)

1.1.2 Modeling Visitor Counts

Similarly, if you plot a histogram of the people counts for each file, you will see that the probability mass function is a Poisson distribution, but each with its own mean.



(Once again: lot of overlap – a classifier based only on the number of visitors in an hour would not be accurate.)

What is the value for λ_D that would make the visitor counts in `depots.csv` most likely?

What is the value for λ_L that would make the visitor counts in `laundromats.csv` most likely?

1.1.3 Modeling Felon-Ownership

Given a shop is a drug depot, what is the probability s_D that it is owned by a felon?

Given a shop is a legit laundromat, what is the probability s_L that it is owned by a felon?

1.2 Learning: making parameters from labeled data

When you run `bc_main.py`, it will read in the labeled data (`depots.csv` and `laundromats.csv`).

For each file, it will call your `make_parameters` function. You will make a dictionary of parameters that are the most likely estimators of training data:

key	meaning
<code>mu_a</code>	The mean of the area in square feet
<code>std_a</code>	The standard deviation of the area. (Not variance!)
<code>lambda_v</code>	The mean of the number of visitors, which are Poisson distributed
<code>s</code>	The probability that the owner is a felon

1.3 Classification

Next, `bc_main.py` will create an instance of your `Classifier` class. It will pass it the priors and the parameters. You have two classes, so they will each be a list of length 2.

Next it will read in unlabeled data from `unlabeled_validate.csv`. For every row in the file, it will call your classifiers `predict` method with a tuple representing the data in the row. You will use Bayes' rule to give two probabilities:

- The probability it is a depot
- The probability it is really a laundromat

They should add up to 1.

You can assume that the three random variables (A, V_1, V_2, F) are conditionally independent given D . That is,

$$p(A, V_1, V_2, F|D) = p(A|D)P(V_1|D)P(V_2|D)P(F|D)$$

Remember the prior: $P(D = \text{depots}) = 0.2$

If $p(D = \text{depots}|A, V_1, V_2, F)$ is greater than 0.5, you will predict that it is a depot.

You will want to do this in "log space". All the standard statistics packages (you will use `scipy.stats`) have the ability to get the log of probability density instead of the probability density. In the constructor, you will convert all the priors to log priors. (Of course, you will bring it into regular "not log space" before you compare it to 0.5.)

When you run `bc_main.py`, it should look like this:

```
> python3 bc_main.py
For depots:
    mu_a 2293.734239
    std_a 507.118758
    lambda_v 20.184783
    s 0.393116
For laundromats:
    mu_a 2898.230290
    std_a 298.293988
    lambda_v 15.697545
    s 0.160714
Reading data/unlabeled_validate.csv
Wrote data/predictions_validate.csv.
```

And `predictions_validate.csv` will include the probability that each row is a depot and the final prediction:

```
shop_id,pdepot,prediction
1001,0.2593,laundromats
1002,0.0016,laundromats
1003,0.6980,depots
...
```

1.4 Evaluating Your Classifier

You will evaluate your classifier (based on the validation data) with a second program, `eval_main.py`. (The correct answers are in .)

You will be editing `evaluator.py`. (You will not need to edit `eval_main.py`.)

Using the ground truth in `labels_validate.csv` and your predictions in `unlabeled_validate.csv`, `eval_main.py` will produce a dictionary:

- The keys will be pairs, where the first term is what you predicted and the second is what it actually was: `("laundromats","laundromats")`, `("laundromats","depots")`, `("depots","laundromats")`, `("depots","depots")`
- The values will be the count of how many times that combination occurred.

You will implement `gather_stats` in `evaluator.py` which will put several statistics in a dictionary

- Total number of items n
- Accuracy
- Recall of depots
- Precision of depots
- F1 score

The output, then, should look something like this (your output will differ slightly):

```
> python3 eval_main.py
Data for confusion matrix:
  Predicted "depots", Actual "depots": 70
  Predicted "depots", Actual "laundromats": 8
  Predicted "laundromats", Actual "depots": 37
  Predicted "laundromats", Actual "laundromats": 135
Total data points: 250
Total correct: 205
Total wrong: 45
Accuracy: 82.0%
```

Recall of depots: 65.4%
Precision of depots: 89.7%
F1: 0.757

2 Hidden Markov Model

This is a completely separate project: Your agent is gambling for you, and the croupier is cheating.

Your code decides whether or not to bet. The croupier rolls one six-sided die. Here is the payoff:

roll	payoff
1	-\$2.50
2	-\$1
3	-\$1
4	-\$1
5	\$0
6	\$5

The croupier has three dice: Fair, Mean, and Kind. Here are probabilities for the six sides are different on the dice:

roll	Fair	Mean	Kind
1	1/6	0.3	0.1
2	1/6	0.4	0.15
3	1/6	0.05	0.1
4	1/6	0.05	0.2
5	1/6	0.05	0.2
6	1/6	0.15	0.25

When he begins, he picks one of the three dies randomly, but weighted. Here is the probability that the first roll will be done with each die: 0.07, 0.9, 0.03.

After each roll, he considers changing the die, here is the transition matrix:

$$\begin{bmatrix} 0.82 & 0.11 & 0.07 \\ 0.15 & 0.78 & 0.07 \\ 0.08 & 0.08 & 0.84 \end{bmatrix}$$

(All this info is the `dice3.json` file that will be read by `hmm_main.py`.)

By watching the rolls, your agent will update its beliefs about which die is being used. When its expected winnings would be greater than zero, your agent will bet.

2.0.1 Gambler.py

To run the program, you run `hmm_main.py` with the name of the dice data file and a number of rolls:

```
> python3 hmm_main.py data/dice3.json 2000000
```

Here is the stationary distribution of die use:


```
[0.38998682 0.30566535 0.30434783]
```

Expected winnings per roll for each die:

```
[-0.08333331 -0.5          0.55          ]
```

Expected per roll winnings betting every roll: \$-0.02

Expected per roll winnings if you knew which die was being used: \$0.17

Expected total winnings if you bet on every roll: \$-35,880.53

Expected total winnings if you knew which die was being used: \$334,782.61

Total would have won if bet on every roll: \$-39,305.50

Total would have won if you knew the die was a winner: \$331,168.50

Did win: \$95,434.50

Average per roll winnings: \$0.04771725

You will implement the following methods in `Gambler.py`:

- a constructor
- Some interesting stats:
 - `stationary_distribution`: returns the stationary distribution of the use of the three dice. It will be a numpy array that sums to 1.0.
 - `expected_winning_by_die`: You should compute this in the constructor and just return it here.
 - `expected_winnings_per_roll_bet_every_roll`: If you bet on every single roll, what you would expect to win on a single roll. (Use the stationary distribution!)
 - `expected_winnings_per_roll_bet_on_good_dice`: If the croupier was your friend and told you when the kind die was in play, how much would you make on average per roll? (Include all the zeros you would get while the fair and mean dice were rolled.)
- The game itself:
 - `will_bet`: returns `True` if your agent thinks it is a good time to bet.
 - `update`: This is how the croupier tells you what was rolled so that you can update your beliefs.

`Gambler.py` should have no loops. None.

Your `update` method should update the beliefs about which die is in use based on the roll. Then it should further update its beliefs based on the transition matrix.

To test your code, you should play at least one hundred thousand rolls. For large numbers of rolls, your per roll winnings should be close to \$0.047.

3 Testing Reminders

Before submitting, make sure each piece works locally:

3.1 Naive Bayesian Classifier

```
> python3 bc_main.py  
> python3 eval_main.py
```

Check out the regenerated `params.pkl`, the predictions, and the evaluation metrics to confirm your results.

3.2 Gambler simulation

After following the earlier instructions to run `hmm_main.py`, try a few different roll counts (e.g., 200000, 500000) and make sure the per-roll winnings stay near \$0.047.

4 Criteria for success

You will submit the following to Gradescope:

- `make_parameters.py` - which will evaluate the training data and produce the parameters.
- `Classifier.py` - which will use the parameters and priors to predict whether a row of data represents a laundromat or a drug depot.
- `evaluator.py` - which will produce statistics on your classifier.
- `Gambler.py` - Your gambling agent.

Do this work by yourself. A hint from another student is OK. Looking at another student's code is *not* OK. Any use of an LLM is forbidden.

You may not use any other libraries than those already imported in the template files.